

FLEXBOX FROGGY

◀ Nivel 1 de 24 ▶

Bienvenido a Flexbox Froggy, un juego donde ayudarás a Froggy y a sus amigos escribiendo código CSS. Guía a esta rana hacia la hoja de lirio en la derecha, usando la propiedad `justify-content`, la cual alinea elementos horizontalmente y acepta los siguientes valores:

- `flex-start`: Alinea elementos al lado izquierdo del contenedor.
- `flex-end`: Alinea elementos al lado derecho del contenedor.
- `center`: Alinea elementos en el centro del contenedor.
- `space-between`: Muestra elementos con la misma distancia entre ellos.
- `space-around`: Muestra elementos con la misma separación alrededor de ellos.

Por ejemplo, `justify-content: flex-end;` moverá la rana a la derecha.

```
1 #pond {
2   display: flex;
3   justify-content: flex-end;
4 }
```

Siguiente

Flexbox Froggy es una creación de [Codepip](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Settings](#)

FLEXBOX FROGGY

◀ Nivel 2 de 24 ▶

Nuevamente, utiliza `justify-content` para ayudar a esas ranas a llegar a sus hojas de lirio. Recuerda que esta propiedad CSS alinea elementos horizontalmente y acepta los siguientes valores:

- `flex-start`: Alinea elementos al lado izquierdo del contenedor.
- `flex-end`: Alinea elementos al lado derecho del contenedor.
- `center`: Alinea elementos en el centro del contenedor.
- `space-between`: Muestra elementos con la misma distancia entre ellos.
- `space-around`: Muestra elementos con la misma separación alrededor de ellos.

```
1 #pond {
2   display: flex;
3   justify-content: center;
4 }
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 3 de 24 ▶

Ayuda a las tres ranas a encontrar sus hojas de lirio usando `justify-content`. En esta oportunidad, las hojas de lirio tienen mucho espacio a su alrededor.

Si no recuerdas los valores de una propiedad, puedes pasar el cursor sobre el nombre de una e inmediatamente se mostrarán. Prueba pasando el cursor sobre `justify-content`.

```
1 #pond {
2   display: flex;
3   justify-content: space-around;
4 }
5
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 4 de 24 ▶

Ahora las hojas de lirio de los bordes se han desplazado a un costado, aumentando así el espacio entre ellas. Usa `justify-content`. En esta oportunidad, las hojas de lirio tienen el mismo espacio entre ellas.

```
1 #pond {
2   display: flex;
3   justify-content: space-between;
4 }
5
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 5 de 24 ▶

Ahora usa `align-items` para ayudar a las ranas a llegar al fondo del estanque. Esta propiedad CSS alinea elementos verticalmente y acepta los siguientes valores:

- `flex-start`: Alinea elementos a la parte superior del contenedor.
- `flex-end`: Alinea elementos a la parte inferior del contenedor.
- `center`: Alinea elementos en el centro (verticalmente hablando) del contenedor.
- `baseline`: Muestra elementos en la línea base del contenedor
- `stretch`: Elementos se estiran para ajustarse al contenedor.

```
1 #pond {
2   display: flex;
3   align-items: flex-end;
4 }
5
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 6 de 24 ▶

Mueve la rana al centro del estanque, usando una combinación de `justify-content` y `align-items`.

```
1 #pond {  
2   display: flex;  
3   justify-content: center;  
4   align-items: center;  
5 }  
6  
7  
8  
9  
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 7 de 24 ▶

Nuevamente, las ranas necesitan cruzar el estanque. En esta oportunidad, las hojas de lirio tienen mucho espacio alrededor de ellas. Debes usar una combinación de `justify-content` y `align-items`.

```
1 #pond {  
2   display: flex;  
3   justify-content: space-around;  
4   align-items: flex-end;  
5 }  
6  
7  
8  
9  
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 8 de 24 ▶

Las ranas necesitan ponerse en el mismo orden que sus hojas de lirio usando `flex-direction`. Esta propiedad CSS define la dirección de los elementos en el contenedor, y acepta los siguientes valores:

- `row`: Elementos son colocados en la misma dirección del texto.
- `row-reverse`: Elementos son colocados en la dirección opuesta al texto.
- `column`: Elementos se colocan de arriba hacia abajo.
- `column-reverse`: Elementos se colocan de abajo hacia arriba.

```
1 #pond {  
2   display: flex;  
3   flex-direction: row-reverse;  
4 }  
5  
6  
7  
8  
9  
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 9 de 24 ▶

Ayuda a las ranas a encontrar su hoja de lirio en la columna usando `flex-direction`. Esta propiedad CSS define la dirección de los elementos en el contenedor, y acepta los siguientes valores:

- `row`: Elementos son colocados en la misma dirección del texto.
- `row-reverse`: Elementos son colocados en la dirección opuesta al texto.
- `column`: Elementos se colocan de arriba hacia abajo.
- `column-reverse`: Elementos se colocan de abajo hacia arriba.

```
1 #pond {
2   display: flex;
3   flex-direction: column;
4 }
5
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 10 de 24 ▶

Ayuda a las ranas a llegar a sus hojas de lirio. A pesar de que ellas parecen estar cerca, necesitarás usar `flex-direction` y `justify-content` para llegar hasta allí.

Nota que cuando estableces la dirección a una fila invertida o columna, el inicio y el final también se invierten.

```
1 #pond {
2   display: flex;
3   flex-direction: row-reverse;
4   justify-content: flex-end;
5 }
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 11 de 24 ▶

Ayuda a las ranas a encontrar sus hojas de lirio usando `flex-direction` y `justify-content`.

Nota que cuando es una columna, `justify-content` cambia a vertical y `align-items` a horizontal.

```
1 #pond {
2   display: flex;
3   flex-direction: column;
4   justify-content: flex-end;
5 }
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 12 de 24 ▶

Ayuda a las ranas a encontrar sus hojas de lirio usando `flex-direction` y `justify-content`.

```
1 #pond {  
2   display: flex;  
3   justify-content: space-between;  
4   flex-direction: column-reverse;  
5 }  
6  
7  
8  
9  
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 13 de 24 ▶

Ayuda a las ranas a encontrar sus hojas de lirio usando `flex-direction`, `justify-content`, y `align-items`.

```
1 #pond {  
2   display: flex;  
3   flex-direction: row-reverse;  
4   justify-content: center;  
5   align-items: flex-end;  
6 }  
7  
8  
9  
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 14 de 24 ▶

A veces, invertir el orden de una fila o columna de un contenedor no es suficiente. En esos casos, nosotros podemos aplicar la propiedad `order` a elementos individuales. Por defecto, los elementos tienen un valor 0, pero nosotros podemos usar esta propiedad para establecerlo a un número entero positivo o negativo.

Usa la propiedad `order` para reordenar las ranas de acuerdo a sus hojas de lirio.

```
1 #pond {
2   display: flex;
3 }
4
5 .yellow {
6   order: 1;
7 }
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 15 de 24 ▶

Usa la propiedad `order` para enviar la rana de color rojo a su hoja de lirio.

```
1 #pond {
2   display: flex;
3 }
4
5 .red {
6   order: -1;
7 }
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 16 de 24 ▶

Otra propiedad que puedes aplicar a elementos individuales es `align-self`. Esta propiedad acepta los mismos valores de `align-items` y sus valores son usados para un elemento específico

```
1 #pond {
2   display: flex;
3   align-items: flex-start;
4 }
5
6 .yellow {
7   align-self: flex-end;
8 }
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 17 de 24 ▶

Combina `order` con `align-self` para ayudar a las ranas a llegar a sus destinos.

```
1 #pond {
2   display: flex;
3   align-items: flex-start;
4 }
5
6 .yellow {
7   order: 2;
8   align-self: flex-end;
9 }
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 18 de 24 ▶

¡Oh, no! Todas las ranas están apretadas en una sola fila de hojas de lirio. Manos a la obra, vamos a distribuir las ranas correctamente usando la propiedad `flex-wrap`, la cual acepta los siguientes valores:

- `nowrap`: Cada elemento se ajusta en una sola línea.
- `wrap`: los elementos se envuelven alrededor de líneas adicionales.
- `wrap-reverse`: Los elementos se envuelven alrededor de líneas adicionales en reversa.

```
1 #pond {
2   display: flex;
3   flex-wrap: wrap;
4 }
5
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 19 de 24 ▶

Ayuda a este ejército de ranas a ordenarse en tres columnas usando una combinación de `flex-direction` y `flex-wrap`.

```
1 #pond {
2   display: flex;
3   flex-direction: column-reverse;
4   flex-wrap: wrap;
5 }
6
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 20 de 24 ▶

Las dos propiedades `flex-direction` y `flex-wrap` son usadas a menudo en conjunto con la propiedad abreviada `flex-flow`, la cual fue creada para combinarlas. Esta propiedad abreviada, acepta un valor de cada una separada por un espacio.

Por ejemplo, puedes usar `flex-flow` para establecer filas y envolverlas.

Trata de usar `flex-flow` para volver a realizar el nivel anterior.

```
1 #pond {
2   display: flex;
3   flex-direction: column;
4 }
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 22 de 24 ▶

Ahora, la corriente ha agrupado las hojas de lirio en la parte inferior. Usa `align-content` para guiar las ranas hasta allí.

```
1 #pond {
2   display: flex;
3   flex-wrap: wrap;
4   align-content: flex-end;
5 }
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 23 de 24 ▶

Las ranas han tenido una fiesta, pero es momento de volver a casa. Usa una combinación de `flex-direction` y `align-content` para llevarlas a sus hojas de lirio.

```
1 #pond {
2   display: flex;
3   flex-wrap: wrap;
4   flex-direction: column-reverse;
5   align-content: center;
6 }
7
8
9
10
```

Siguiente

FLEXBOX FROGGY

◀ Nivel 24 de 24 ▶

Trae a las ranas a su casa una vez más usando las propiedades CSS que aprendiste:

- `justify-content`
- `align-items`
- `flex-direction`
- `order`
- `align-self`
- `flex-wrap`
- `flex-flow`
- `align-content`

```
1 #pond {
2   display: flex;
3   flex-wrap: wrap-reverse;
4   flex-direction: column-reverse;
5   align-content: space-between;
6   justify-content: center;
7 }
8
9
10
```

Siguiente

GRID GARDEN

◀ Nivel 1 de 28 ▶

¡Bienvenido a Grid Garden, donde escribirás tu código CSS para cultivar tu jardín de zanahorias! Riega solo las áreas que tienen zanahorias usando la propiedad `grid-column-start`.

Por ejemplo, `grid-column-start: 3;` regará el área comenzando por la tercera línea vertical, que es otra manera de decir el 3er borde vertical contando desde la izquierda de la cuadrícula.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 3;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 2 de 28 ▶

Oh oh, parece que hay malas hierbas creciendo en la esquina de tu jardín. Usa `grid-column-start` para envenenarlas. Fíjate en que las malas hierbas comienzan en el quinto borde vertical de la cuadrícula.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #poison {
8   grid-column-start: 5;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 3 de 28 ▶

Cuando `grid-column-start` se usa solo, la expansión por defecto del elemento en la cuadrícula será de exactamente una columna. No obstante, puedes extender el elemento varias columnas añadiendo la propiedad `grid-column-end`.

Usando `grid-column-end`, riega todas las zanahorias evitando que se forme barro. ¡No queremos malgastar agua! Fíjate en que las zanahorias comienzan en el 1er borde vertical y terminan en el 4º.

Define la posición final de un elemento respecto a las columnas de la cuadrícula.

```
1 #
2
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 1;
9   grid-column-end: span 3 ;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 4 de 28 ▶

Al emparejar `grid-column-start` y `grid-column-end`, podrías asumir que el valor final tiene que ser mayor que el valor inicial. ¡Pero no es el caso!

Intenta establecer `grid-column-end` a un valor inferior a 5 para regar tus zanahorias.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 5;
9   grid-column-end: 2;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 5 de 28 ▶

Si prefieres contar las líneas de la cuadrícula comenzando por la derecha, puedes dar a `grid-column-start` y `grid-column-end` valores negativos. Por ejemplo, puedes establecerlos a -1 para indicar la primera línea comenzando por la derecha.

Intenta establecer `grid-column-end` a un valor negativo.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 1;
9   grid-column-start: span 4;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 6 de 28 ▶

Ahora intenta establecer `grid-column-start` a un valor negativo.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #poison {
8   grid-column-start: 4;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 7 de 28 ▶

En lugar de definir un elemento en la cuadrícula basado en la posición inicial y final, puedes definirlo basado en la longitud de columnas deseada usando la palabra clave `span`. Ten presente que `span` solo funciona con valores positivos.

Por ejemplo, riega las zanahorias usando `grid-column-end: span 2;`.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 2;
9   grid-column-end: span 2;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 8 de 28 ▶

Intenta usar `grid-column-end` con la palabra clave `span` de nuevo para regar tus zanahorias.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 1;
9   grid-column-start: span 5;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 9 de 28 ▶

También puedes usar la palabra clave `span` con `grid-column-start` para establecer la anchura del elemento en relación a la posición final.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 3;
9   grid-column-end: 6;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 10 de 28 ▶

Escribir ambos `grid-column-start` y `grid-column-end` cada vez puede resultar cansador. Afortunadamente, `grid-column` es una propiedad abreviada que acepta ambos valores a la vez, separados por una barra oblicua.

Por ejemplo, `grid-column: 2 / 4;` establecerá el comienzo del elemento de la cuadrícula en la 2ª línea vertical de esta, y su final en la 4ª línea vertical.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 4 / 6;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 11 de 28 ▶

Intenta usar `grid-column` para regar las zanahorias. La palabra clave `span` también funciona con esta propiedad abreviada así que ¡dale una oportunidad!

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 2 / 5;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 12 de 28 ▶

Una de las cosas que diferencia las cuadrículas de CSS de flexbox es que puedes posicionar los elementos fácilmente en 2 dimensiones: columnas y filas. `grid-row-start` funciona de manera semejante a `grid-column-start` pero a lo largo del eje vertical.

Usa `grid-row-start` para regar las zanahorias.

```
1 #garden {  
2   display: grid;  
3   grid-template-columns: 20% 20% 20% 20% 20%;  
4   grid-template-rows: 20% 20% 20% 20% 20%;  
5 }  
6  
7 #water {  
8   grid-row-start: 3;  
9 }  
10  
11  
12  
13  
14
```

Siguiente

GRID GARDEN

◀ Nivel 13 de 28 ▶

Ahora dale una oportunidad a la propiedad abreviada `grid-row`.

```
1 #garden {  
2   display: grid;  
3   grid-template-columns: 20% 20% 20% 20% 20%;  
4   grid-template-rows: 20% 20% 20% 20% 20%;  
5 }  
6  
7 #water {  
8   grid-row: 3/6;  
9 }  
10  
11  
12  
13  
14
```

Siguiente

GRID GARDEN

◀ Nivel 14 de 28 ▶

Usa `grid-column` y `grid-row` a la vez para establecer una posición en ambas dimensiones.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #poison {
8   grid-row: 5;
9   grid-column: 2;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 15 de 28 ▶

También puedes usar `grid-column` y `grid-row` juntos para abarcar áreas más extensas en la cuadrícula. ¡Inténtalo!

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 2 / 6;
9   grid-row: 1 / 6;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 16 de 28 ▶

Si escribir `grid-column` y `grid-row` se te hace demasiado pesado, aquí tienes otra propiedad abreviada. `grid-area` admite cuatro valores separados por barras oblicuas: `grid-row-start`, `grid-column-start`, `grid-row-end`, seguido de `grid-column-end`.

Un ejemplo de esto podría ser `grid-area: 1 / 1 / 3 / 6;`.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-area: 1/2/4/6;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 17 de 28 ▶

¿Y qué me dices de múltiples elementos? Puedes superponerlos sin problema. Usa `grid-area` para definir una segunda área que cubra todas las zanahorias que están sin regar.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water-1 {
8   grid-area: 1 / 4 / 6 / 5;
9 }
10
11 #water-2 {
12   grid-area: 2/3/5/6;
13 }
14
```

Siguiente

GRID GARDEN

◀ Nivel 18 de 28 ▶

Si los elementos de la cuadrícula no se sitúan explícitamente con `grid-area`, `grid-column`, `grid-row`, etc., se sitúan automáticamente de acuerdo al orden en el código fuente. Puedes sobrescribir esto usando la propiedad `order`, que es una de las ventajas de la cuadrícula frente al diseño basado en tablas.

Por defecto, el valor de `order` de todos los elementos es igual a 0, pero puede ser establecido a cualquier valor positivo o negativo, de manera similar a `z-index`.

Ahora mismo, las zanahorias en la segunda columna están siendo envenenadas y las malas hierbas en la última columna, regadas. ¡Cambia el valor de `order` del veneno para solucionar esto!

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 .water {
8   order: 0;
9 }
10
11 #poison {
12   order: 1;
13 }
14
```

Siguiente

GRID GARDEN

◀ Nivel 19 de 28 ▶

Ahora el veneno y el agua se alternan, a pesar de que todas las malas hierbas están al comienzo de tu jardín. Cambia el valor de `order` de los venenos para solucionar esto.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 .water {
8   order: 0;
9 }
10
11 .poison {
12   order: -1;
13 }
14
```

Siguiente

GRID GARDEN

Hasta este momento, has tenido un jardín formado por cinco columnas, cada una ocupando el 20% de la anchura total, y cinco filas, cada una ocupando el 20% de la altura total.

Esto ha sido establecido con las propiedades `grid-template-columns: 20% 20% 20% 20% 20%;` y `grid-template-rows: 20% 20% 20% 20% 20%;`. Cada propiedad tiene cinco valores que crean cinco columnas, cada una establecida al 20% de la anchura total del jardín.

Pero puedes establecer los valores en la cuadrícula como quieras. Da a `grid-template-columns` un nuevo valor para regar tus zanahorias. Querrás que la anchura de la primera columna sea del 50%.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 50% 50%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 1;
9   grid-row: 1;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 21 de 28 ▶

Especificar un puñado de columnas con la misma anchura puede ser aburrido. Afortunadamente hay una función `repeat` que te ayudará con eso.

Por ejemplo, previamente hemos definido cinco columnas al 20% de anchura mediante `grid-template-columns: 20% 20% 20% 20% 20%`. Esto puedes simplificarlo como `grid-template-columns: repeat(5, 20%);`

Usando `grid-template-columns` con la función `repeat`, crea ocho columnas, cada una con una anchura del 12.5%. De esta manera no inundarás tu jardín.

```
1 #garden {
2   display: grid;
3   grid-template-columns: repeat(8, 12.5%);
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 1;
9   grid-row: 1;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 22 de 28 ▶

`grid-template-columns` no acepta solo valores porcentuales, sino también otras unidades como pixels y ems. Incluso puedes mezclar diferentes unidades a la vez.

Establece tres columnas a `100px`, `3em`, y `40%` respectivamente.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 100px 3em 40%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7
8
9
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 23 de 28 ▶

CSS Grid también introduce una nueva medida, la fracción `fr`. Cada unidad `fr` asigna una porción del espacio disponible. Por ejemplo, si dos elementos están establecidos a `1fr` y `3fr` respectivamente el espacio se divide en 4 porciones iguales; el primer elemento ocupa 1/4 del espacio y el segundo elemento los 3/4 restantes.

Ahora las malas hierbas ocupan 1/6 de tu primera fila y las zanahorias los restantes 5/6. Crea dos columnas con ese ancho empleando `fr`.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 1fr 5fr;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7
8
9
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 24 de 28 ▶

Cuando algunas columnas son establecidas en píxeles, porcentajes o ems, cualquier otra columna establecida con `fr` dividirá el espacio restante.

Aquí las zanahorias forman una columna de 50 píxeles a la izquierda, y las malas hierbas una columna de 50 píxeles a la derecha. Mediante `grid-template-columns`, crea esas dos columnas y usa `fr` para crear tres columnas más en el espacio que queda entre ellas.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 50px repeat(3, 1fr) 50px;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-area: 1 / 1 / 6 / 2;
9 }
10
11 #poison {
12   grid-area: 1 / 5 / 6 / 6;
13 }
14
```

Siguiente

GRID GARDEN

◀ Nivel 25 de 28 ▶

Ahora hay una columna de malas hierbas de 75 píxeles en el lado izquierdo del jardín. En 3/5 del espacio restante crecen zanahorias, mientras que los 2/5 restantes han sido invadidos por malas hierbas.

Usa `grid-template-columns` con una combinación de `px` y `fr` para crear las columnas necesarias.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 75px 3fr 2fr;
4   grid-template-rows: 100%;
5 }
6
7
8
9
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 26 de 28 ▶

`grid-template-rows` funciona exactamente igual que `grid-template-columns`.

Usa `grid-template-rows` para regar todo excepto los 50 píxeles de la parte superior de tu jardín. Fíjate que el agua se ha establecido para que llene solo la 5ª fila, por lo que tendrás que crear 5 filas en total.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 50px 0 0 0 1fr;
5 }
6
7 #water {
8   grid-column: 1 / 6;
9   grid-row: 5 / 6;
10 }
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 27 de 28 ▶

`grid-template` es una propiedad abreviada que combina `grid-template-rows` y `grid-template-columns`.

Por ejemplo, `grid-template: 50% 50% / 200px;` creará una cuadrícula con dos filas que ocuparán el 50% del alto cada una, y una columna que será 200 píxeles de ancho.

Prueba a usar `grid-template` para regar un área que incluya el 60% superior y 200 píxeles desde la izquierda en tu jardín.

```
1 #garden {
2   display: grid;
3   grid-template: 60% / 200px;
4 }
5
6 #water {
7   grid-column: 1;
8   grid-row: 1;
9 }
10
11
12
13
14
```

Siguiente

GRID GARDEN

◀ Nivel 28 de 28 ▶

Tu jardín tiene una pinta genial. Esta vez se ha dejado un camino de 50 píxeles de ancho en el fondo de tu jardín y se ha llenado el resto con zanahorias.

Desafortunadamente, el 20% izquierdo de tus zanahorias han sido invadidas por malas hierbas. Usa CSS Grid una última vez para tratar tu jardín.

```
1 #garden {
2   display: grid;
3   grid-template: 1fr 50px / 20% 80%;
4 }
5
6
7
8
9
10
11
12
13
14
```

Siguiente